claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-aadbbf6b486b15018.jsonl`
- SHA-256: `0ac7f5178532b25ccd8df34dd6eddbaca03e59dd9dd795b7a541ecdb1733b799`
- Source modified: `2026-06-21T13:58:32+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:55:40.906Z)

```
Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app.
For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether
it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot
confirm. Use web search to check current facts.
TOPIC: distribution-channels
CLAIMS:
[
  {
    "claim": "torch/torchvision are deliberately omitted from [project.dependencies] because
their CPU/CUDA wheels need --index-url/--extra-index-url, which PEP 621 cannot express; the app
already documents installing them on a separate pip line (cu124 or cpu).",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml:8-11,58-64"
  },
  {
    "claim": "console script is `indexer = backend.main:main`; build backend is hatchling;
backend is a namespace package so wheel packages are listed explicitly as [backend, training];
requires-python>=3.10; extras = dev/reporting(git+ssh)/gpu(empty)/storage-s3/storage-gcs.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml:21,72-88"
  },
  {
    "claim": "Standard PEP 508 dependency specs cannot carry a custom index URL; pip has an open
feature request (#13000) to allow --extra-index-url in pyproject.toml, confirming the limitation
is real and unresolved.",
    "confidence": "high",
    "source": "https://github.com/pypa/pip/issues/13000"
  },
  {
    "claim": "uv's PyTorch index handling (tool.uv.index with explicit=true + tool.uv.sources +
platform markers, and the --torch-backend=auto/cpu/cuNNN flag that auto-detects the GPU driver)
is the cleanest solution to the torch index problem, but --torch-backend and tool.uv index
config ONLY work in the `uv pip` / project (lock/sync/run) interface ? NOT in `uv tool install`
or `uvx`. Requires uv >=0.5.3.",
    "confidence": "high",
    "source": "https://docs.astral.sh/uv/guides/integration/pytorch/"
  },
  {
    "claim": "uv installs as a standalone binary on Windows via one PowerShell line (`irm
https://astral.sh/uv/install.ps1 | iex`), auto-configures PATH, and can download the required
Python itself ? so the user does not need Python pre-installed; pipx by contrast uses whichever
Python is already present and requires manually adding its scripts dir to PATH.",
    "confidence": "high",
    "source": "https://docs.astral.sh/uv/getting-started/installation/"
```

```
  },
  {
    "claim": "pipx can install from a private index/direct wheel and route torch's index via
`--pip-args='--index-url ...'` or PIP_INDEX_URL, and `pipx inject <app> torch --pip-args=...`
adds torch into the app's isolated venv post-install.",
    "confidence": "high",
    "source": "https://github.com/pypa/pipx/issues/1009"
  },
  {
    "claim": "Offline install is standard via `pip download -d wheelhouse` (with --only-
binary=:all:/--platform/--python-version to target the destination interpreter) then `pip
install --no-index --find-links wheelhouse <pkgs>`; you must run pip download from the SAME
Python minor version as the target machine.",
    "confidence": "high",
    "source": "https://pip.pypa.io/en/stable/cli/pip_download/"
  },
  {
    "claim": "PyInstaller bundling torch+CUDA yields 2.6GB (onefile, pip cu118) to ~5GB
artifacts and is fragile with hidden imports; CUDA/cuDNN shared libs cannot be shrunk ? making a
single-exe with real GPU CV impractical.",
    "confidence": "high",
    "source": "https://github.com/orgs/pyinstaller/discussions/8552"
  },
  {
    "claim": "The live appliance design (owner decision D10) is a self-hosted webserver+UI the
user spins up (NOT a desktop app), bound to 127.0.0.1:8000, driving the existing FastAPI loop
and `python -m backend.worker {infer|train}` dispatcher; storage=local default; compute seam
detector_impl ? {auto=WSL, native, stub}.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:12-46 and
backend/main.py:638, backend/worker/__main__.py:1-15"
  },
  {
    "claim": "The hard packaging problem ? native WASB shuttle model needs torch 1.11+cu113 in a
SEPARATE conda env while the main engine uses torch 2.6+cu124 ? cannot be solved by ANY single-
venv installer (pip/pipx/uv tool) nor by a single PyInstaller bundle; it requires a second
managed environment or container.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:96 (native
WASB conda env makes the image non-trivial)"
  }
]
```


RECOMMENDATION UNDER TEST: Lead with (c) uv tool install, fronted by a generated one-line install script, and back it with (d) a private/direct wheel source; keep (a) PyPI-wheel+pip as the documented power-user fallback and ship (e) an offline CPU bundle for the field/air-gapped persona. Reasoning: the dominant non-tech-Windows barrier is "does the user have a correct Python and can they type the torch --index-url line." uv is the only channel that (1) installs itself in one PowerShell line with automatic PATH, (2) downloads a correct Python 3.10+ so the user needs nothing pre-installed, and (3) is fast and per-app isolated. Because `--torch-backend=auto` and `tool.uv.index` do NOT work in `uv tool install`/`uvx` (they are `uv pip`/project-only), do NOT rely on `uv tool install <app>` to also fetch torch. Instead ship a thin generated installer (install-indexer.ps1) that: (i) bootstraps uv, (ii) runs `uv tool install --from <your-wheel-url-or-private-index> badminton-highlight-indexer`, (iii) installs the matching torch into that tool's environment using `uv pip install --torch-backend=auto torch torchvision` against the tool's venv (or, in a uv project layout, sets tool.uv.index+sources and runs uv sync), (iv) drops ffmpeg/ffprobe next to the venv and exposes them on PATH, (v) on first `indexer --server` run fetches wasb_badminton_best.pth.tar from GCS. The script makes the torch index-url problem invisible ? the user double-clicks one .ps1. Do NOT pursue PyInstaller onefile for the GPU path (2.6-5GB, brittle, still can't host the WASB torch-1.11 conda env). The WASB second-env stays a separate concern: gate real local-GPU detection behind a SEPARATE managed env (conda/uv venv #2) or a container, and ship the truly-local default as Gemini/stub/CPU-torch so the one-click install always yields a working appliance even on a CPU-only Windows box.

2. assistant (2026-06-21T13:55:44.243Z)

I'll verify each claim adversarially. Let me start by reading the actual project files referenced, then check the external facts via web search.

3. user (2026-06-21T13:55:45.213Z)

```
1       # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2       #
3       # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4       # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5       # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6       # auto-detection would otherwise miss the un-__init__'d `backend` root.
7       #
8       # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9       # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10      # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11      # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12      [build-system]
13      requires = ["hatchling"]
14      build-backend = "hatchling.build"
15
16      [project]
17      name = "badminton-highlight-indexer"
18      version = "0.1.0"
19      description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20      readme = "README.md"
21      requires-python = ">=3.10"
22      license = { text = "MIT" }
23      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24      keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26      # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27      dependencies = [
28          "fastapi>=0.111.0",
29          "uvicorn>=0.30.1",
30          "opencv-python>=4.9.0.80",
31          "numpy>=1.26.4",
32          "pydantic>=2.7.1",
33          "jinja2>=3.1.4",
34          "python-dotenv>=1.0.0",
35          "google-genai>=2.4.0",
36          "supervision>=0.21.0,<0.30.0",
37      ]
38
39      [project.optional-dependencies]
40      # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41      dev = [
42          "pytest",
43          "pytest-cov",
44          "httpx",        # fastapi TestClient transport
45          "ruff",
46          "mypy",
47      ]
48
49      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50      # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51      # fetched over SSH; if you lack access, simply omit this group ? reporting
52      # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53      #   pip install -e ../sports-obsreport --no-build-isolation
54      reporting = [
55          "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56      ]
57
58      # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59      # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
```

```
60      # express, so they are NOT declarable here. Install them out-of-band instead:
61      #   # CUDA 12.4 box:
62      #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63      #   # CPU-only box (also what CI uses):
64      #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65      gpu = []
66
67      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70      # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71      # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72      storage-s3 = ["boto3>=1.34"]
73      storage-gcs = ["google-cloud-storage>=2.16"]
74
75      [project.scripts]
76      # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77      indexer = "backend.main:main"
78
79      [tool.hatch.metadata]
80      # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81      # reference*); hatchling requires opting in to allow those in metadata.
82      allow-direct-references = true
83
84      [tool.hatch.build.targets.wheel]
85      # `backend` is a namespace package (no __init__.py); list both import roots
86      # explicitly so an editable/wheel install exposes `import backend` and
87      # `import training` regardless of cwd.
88      packages = ["backend", "training"]
89
```

4. assistant (2026-06-21T13:55:48.741Z)

The pyproject.toml confirms claims (a) and (b) in detail. Now let me read the other referenced files and verify the external facts in parallel.

5. user (2026-06-21T13:55:50.615Z)

```
1       # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3       > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Tracking anchors:** §9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6       > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7       > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9       ---
10
11      ## 0. TL;DR
12      **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
```

profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the degenerate collapse of the same storage/worker split.

The **single most important caveat:** the engine has **zero app-layer auth and CORS `*`** `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`, and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the tenancy boundary.**

Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ? provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel** with the UI MVP, which ships on Gemini today.

1. Problem & goal

The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker {infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole loop on a single box, **honestly, behind a gate**.

Headline CUJ (owner's words): a **hosted website** where I pick an input ? **a Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload** ? it lands in the **bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host** running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**

Outcome ("good"): an operator signs in, picks a source, watches it index, curates rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host later** (native WASB) with **no UI fork**.

Read to get current: `PER_RALLY_SELECTION.md` (§3/§4/§10 contract anchors), engine `HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built), `VIDEO_LOCALITY_MODEL.md` (privacy/retention).

2. Decisions locked

#	Decision	Source / date	Implication
D1	Reverse-proxied **single origin**; engine stays `127.0.0.1:8000`	`[verified main.py:638]`; 2026-06-20	CORS `*` is moot from the browser ? the proxy is the boundary
D2	**Edge auth**, zero engine auth code	`HOSTING_PLAN.md`; 2026-06-20	Caddy basicauth OR cloudflared + Cloudflare Access
D3	MVP drives the **in-process FastAPI worker** loop, `storage=local`	`[verified job_worker.py:37, local.py:33]`; 2026-06-20	bucket worker = reserved scale path, out of MVP
D4	Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim**	PR [#11]; 2026-06-20	the console is a superset; do not redesign the picker
D5	Console lives in the khelsutra **`web/`** React scaffold	`web/README.md`; 2026-06-20	Cloudflare-Pages-style root, not engine static
D6	Compute placement is a **setting**, rendered from `GET /api/capabilities`	`[design]`; 2026-06-20	UI offers ONLY what the box can honestly do
D7	**Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to one video object	`[verified backend/storage/ref.py parse_uri]`; 2026-06-20	Drive is a **stub** today ? scoped honestly (§4.2)
D8	GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage stays the bucket	Owner 2026-06-20	compute and storage are separate planes; no provider lock-in
D9	**Observability is first-class** ? structured logs on every new path + a DoD item	Owner 2026-06-20	see §5; verified during CUJ replay
D10	Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the tool	Owner 2026-06-20	**supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings

3. Foundation ? what the engine gives us today `[verified this session unless noted]`

- **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}` (FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT /api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}` `[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-wiring only**, no engine change.
- **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)` `[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a **separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1 to serialize) ? it is **not** already 1.
- **Storage:** `storage.backend` default = `"local"` `[verified models.py:298]`; `LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on `src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://` (AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub (emulated only)**. Credentials never in config ? boto3 default chain / `~/.aws` `[verified s3.py]`.
- **Compute seam:** `indexing.detector_impl` ? {`auto`=WSL, `native`=Linux-native, `stub`=CPU mock}; default `"auto"` `[verified models.py:129]`, resolved in `trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch `feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`; **`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified :57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ? not exercisable on the droplet or in the offline suite. Treat native as **not production-verified**.
- **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ? pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` + `--set indexing.skip_ai_handoff=true` + `storage.s3`.
- **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET /api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}` `[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
- **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md, DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run there.
- **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row `get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.

4. Design / architecture

4.1 Topology & request path

ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy / static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI `127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing site stays a **separate** systemd unit on a **separate** subdomain.

```
PUBLIC INTERNET
   ?  (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
   ?      cloudflared Tunnel + Cloudflare Access (no inbound ports)  OR  Caddy :443 + basicauth
   ?
????????????????????????????????????????????????????????????????????????????????????
? ONE BOX   firewall BLOCKS inbound 8000 + 9000/9001/9023                          ?
? (2) reverse proxy / static host (Caddy/cloudflared)                              ?
?      /, /assets/*  ? web/ React SPA      /api/*, /api/highlights/* ? :8000 ?
?            ?  ONE ORIGIN to the browser ? engine CORS '*' is irrelevant    ?
```

```
66      ?           ?                                                          ?
67      ? (3) ENGINE FastAPI  (host=127.0.0.1:8000, HARD-bound [main.py:638])      ?
68      ?      control: jobs table + ThreadPoolExecutor(max_workers=1) drain        ?
69      ?      data: uploads.py chunked upload · /api/compile · /api/highlights     ?
70      ?           ?                                                          ?
71      ? (4) STORAGE = LocalStorage (default)  fetch=identity · put=no-op(src==dst) ?
72      ?           ?                                                          ?
73      ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]         ?
74      ?                   native WASB (GPU host) · auto (owner Win+WSL)           ?
75      ????????????????????????????????????????????????????????????????????????????
76      RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77         ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ?
outputs/<vid>/ back
78         (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79      ```
80
81      ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82      All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a
`StorageRef` (scale):
83
84      | Source | Path today | Status |
85      |---|---|---|
86      | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3://`/`gcs://`.) | **wire client only** (endpoints exist)
|
87      | **Bucket URI** (`s3://`, `gs://`) | **`POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88      | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90      ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91      | Placement | Runs | Needs | Real/mock | Honest caveat |
92      |---|---|---|---|---|
93      | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94      | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97      | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
```

4.4 Auth / exposure (non-negotiable invariant)
No engine auth, by design; the gate is external. **Invariant (all three must hold):** engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist `[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**; set `security.allowed_video_root`; rate-limit; per-user quota.

4.5 UI operator console (web/ SPA ? superset of RallyPicker)
- **S1 Ingest & Process** ? the source menu (§4.2) + a segmenter picker from `/api/capabilities` ? `POST /api/process`.
- **S2 Progress** ? 3s poll of the free-text stage (`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-failure terminal state.
- **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`): honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter. **Per-rally is ONE screen of the console.**
- **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for **reload-survival**.
- **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.

4.6 Reuse map
Reused (no re-plumb): the full `/api` loop; the in-process worker; the chunked upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing `NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the `HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units; `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker bridge.

5. Observability ? logging & monitoring `[required, first-class]`
Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously, with a full log-analysis pass across every new code path.** The engine already uses a centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new path.

- **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`) **plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ? GPU host. Log it on every line.
- **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry / result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.
- **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must **never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
- **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a ledger exists) for the engine-mode banner + alerting.
- **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is **ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the instance.
- **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay** (§8 Layers 1?3). Ties [[feedback-launch-quality-bar]].

6. Risk table
Risk	Today	Mitigation
No app auth + CORS `*` `[main.py:42]`	open if any of {localhost-bind, firewall, gate} slips	enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
Test ports on shared droplet	MinIO `9000/9001` + GCS emu `9023`	firewall closes them
Retention	`output/`+`uploads/` originals **never deleted**	sweep / keep-proxy-only before any non-owner exposure

Surprise Gemini/GPU bill	**no cost ceiling/ledger**	per-video cap; `ai_max_workers` (default 5) tunable; `max_workers=1` job serialization
Native WASB unverified	partially wired; owner-parity-gated	treat as not-production-verified until owner byte-parity on a real GPU box
New engine endpoints	`capabilities`/`videos`/`reels`/`health` don't exist	coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD, explicit `git add`**
Drive ingest overstated	`gdrive://` is a stub	don't present Drive as live; require Drive?bucket pre-step until real
No Dockerfile	runbook-only reproducibility	feasibility-check a worker image before promising Cloud Run/GCE
Shared-checkout collisions	active worker sessions share the engine tree	the branch-safety protocol on every engine-side PR
Provisioning-credential leak	a DO API token / cloud creds could leak if pasted in chat or committed (cf. the **already-rotated Spaces key** exposed in chat, `DECOUPLED_COMPUTE/HANDOFF.md`)	authenticate the CLI **on-box** (`doctl auth init` interactive / `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token, revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action every time**

7. Honest limits ? what this does NOT do

Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not** containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the **console plumbing**, not engine rally-detection quality (a separate track).

8. CUJs & live-test plan `[required]`

CUJs: **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core); **CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters + honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).

Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:

- **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure; `query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ? CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`). The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug (CUJ-3), capability honesty (CUJ-4).
- **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main --server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema, compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock handler **and** a real-engine contract test **in the same PR**. Runs in CI.
- **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1 with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption; reuse the `parity.yml` dual-host pattern.
- **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job + headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no stacking**; coverage at/above the floor; **logs inspected during replay** (§5).
- **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg` behavior; a multi-GB browser upload. Reported back into §9, stated plainly as **not verified** in the offline suite.

9. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR ships its tests + meaningful logs (§5).

ID	Deliverable	Depends on	Gated?	Status	PR
T0	This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers	?	owner	?	(this PR)
T1	Mock engine + capability fixtures + browser-replay CUJ scripts (L1)	T0	no	?	?
T2	Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key (P1)	?	owner	?	?
T3	L3 live posture test (raw `:8000`/`:900x` refused from outside)	T2	no	?	?
T4	S1 ingest (local upload wired) + segmenter picker	T1	no	?	?
T5	S2 progress (poll + one-job + restart-failure)	T4	no	?	?
T6	S3 embed **RallyPicker** + compile + download	T4	no	?	?
T7	*(engine)* `GET /api/capabilities` + `GET /api/health` + banner	T1	coord	?	?
T8	*(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival)	?	coord	?	?
T9	S5 Settings (config+capabilities; disabled compute/storage target)	T7,T8	no	?	?
T10	Observability pass: correlation id end-to-end + structured logs + metrics (§5)	T4	no	?	?
T11	Retention sweep for `output/`+`uploads/`	?	owner	?	?
T12	Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE** (`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (= **B-P2**)	T4	coord	?	engine done; SPA pending
T13	**GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner byte-parity (P4)	branch	? owner	?	?
T14	*(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5)	T8,T12	? owner	?	?

Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP does **not** block on it (ships on Gemini).

10. Open questions ? owner / external

Blocking gates: (1) **edge-auth** ? Caddy basicauth (self-contained) vs cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention defaults** ? auto-delete originals after compile? keep proxies+intervals only? window? (privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4) **GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration endpoint + how it coordinates with the active worker sessions.

Nice-to-knows: console home (web/ vs engine static); a review-playback streaming endpoint; per-user scoping timing; containerization vs runbook.

11. Definition of done

- [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via the ingest menu.
- [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
- [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped honestly (not faked).
- [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
- [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via `/api/videos` + `/api/reels`.
- [ ] **Observability:** correlation id threaded end-to-end; structured logs at every new path; loud failures; logs inspected during CUJ replay.
- [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
- [ ] CI green (L2 contract + L4 regression); coverage ? floor; §9 tracker all ?/?-resolved.

12. References

Engine code `[verified this session]`:
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;

`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
191
192   ---
193   ### Changelog
194   - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
re-verified against source before writing (catching the native-WASB partial-wiring +
`ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
multi-provider GPU, and first-class Observability directives.
195

6. user (2026-06-21T13:55:58.843Z)

Web search results for query: "uv tool install torch --torch-backend pytorch index not supported
uv pip only"

Links: [{"title":"Using uv with PyTorch |
uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"},{"title":"How to Install
PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-
pytorch-with-uv/"},{"title":"Managing PyTorch CPU/CUDA Versions per Environment with uv Part 2 |
MJUN Tech Note","url":"https://mjunya.com/en/posts/2024-10-22-python-uv-
pytorch-2/"},{"title":"Add documentation for installing common pytorch variants · Issue #5945 ·
astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/5945"},{"title":"Project Setup for
Machine Learning with PyTorch and UV | by Florian Trautweiler |
Medium","url":"https://medium.com/@florian-trautweiler/project-setup-for-machine-learning-with-
pytorch-and-uv-8e3bbb37eb5e"},{"title":"dependencies - A Developer's Guide to Portable PyTorch
Installation with uv and pip -
portability","url":"https://iifx.dev/en/articles/460177044/a-developer-s-guide-to-portable-
pytorch-installation-with-uv-and-pip"},{"title":"pytorch setup with uv | mike
bommarito","url":"https://michaelbommarito.com/wiki/programming/languages/python/pytorch-
setup/"},{"title":"Does auto option in ``torch-backend`` preview only select the highest
supported CUDA version? · Issue #12357 · astral-sh/uv","url":"https://github.com/astral-
sh/uv/issues/12357"},{"title":"GitHub - BaoNguyen6742/uv-install-torch: Tutorial to install
torch/pytorch with cuda using uv · GitHub","url":"https://github.com/BaoNguyen6742/uv-install-
torch"},{"title":"Installing PyTorch: An index URL could not be queried · Issue #8344 · astral-
sh/uv","url":"https://github.com/astral-sh/uv/issues/8344"}]

Based on the search results, I found relevant information about installing PyTorch with uv. Here
are the key findings:

Important Limitation

The `--torch-backend` flag is only available in the `uv pip` interface and does not work with
`uv lock`, `uv sync`, or `uv run`. This means if you're trying to use `uv tool install` with
`--torch-backend`, that command is not supported.

Solution for uv tool

Since `uv tool install` doesn't support the `--torch-backend` flag, you need to use `uv pip
install` instead:

The uv pip interface offers a `--torch-backend` flag that selects the correct PyTorch index
without any pyproject.toml configuration, and setting `--torch-backend=auto` makes uv detect the
available GPU hardware and pick the appropriate backend.

Correct Commands

```
For command-line tool installation with PyTorch:
- Use `uv pip install torch --torch-backend=pytorch` (or your preferred backend)
- Or use `uv pip install torch --torch-backend=auto` to auto-detect your GPU

Valid values for `--torch-backend` are: auto, cpu, cu118, cu126, cu128, cu130, rocm6, xpu.

If you need to configure PyTorch in a project context, you'll need to use `pyproject.toml`
configuration with custom index definitions instead of relying on command-line flags.


REMINDER: You MUST include the sources above in your response to the user using markdown
hyperlinks.
```

7. assistant (2026-06-21T13:56:03.648Z)

```
Both local files confirm claims (a), (b), (h), and (i) closely. Note one nuance: the search
result says `--torch-backend=auto` selects among CUDA versions but I need to verify whether it
also detects "the GPU driver" and whether the claim "tool.uv.index... ONLY work in uv
pip/project" is accurate. Let me fetch the authoritative uv PyTorch doc and verify the remaining
external claims in parallel.
```

8. user (2026-06-21T13:56:10.336Z)

Summary of uv PyTorch Features

(1) The `--torch-backend` Flag

```
**Interfaces supported:** Only the `uv pip` interface.

> "At present, `--torch-backend` is only available in the `uv pip` interface."

The flag does not work with `uv lock`, `uv sync`, `uv run`, `uv tool install`, or `uvx`.

**Auto-detection capability:** When set to `auto`, uv detects hardware and selects an
appropriate index.

> "When enabled, uv will query for the installed CUDA driver, AMD GPU versions, and Intel GPU
presence, then use the most-compatible PyTorch index for all relevant packages"

**Valid values:** The page demonstrates `--torch-backend=auto` and `--torch-backend=cu130`, with
references to other CUDA versions and accelerator types (cpu, rocm, xpu) elsewhere in the
documentation.
```

(2) Index Configuration

```
Using `tool.uv.index` with `explicit = true` and `tool.uv.sources` allows routing PyTorch
packages to specific indexes:

> "We recommend the use of `explicit = true` to ensure that the index is _only_ used for
`torch`, `torchvision`, and other PyTorch-related packages"

This setup prevents generic dependencies from being sourced from the PyTorch index.
```

(3) Minimum uv Version

```
> "Some of the features outlined in this guide require uv version 0.5.3 or later."
```

9. user (2026-06-21T13:56:12.884Z)

Issue Summary

Title: "Add --extra-index-url option to pyproject.toml to install requirements.txt"

Status: Closed as not planned (marked as duplicate and needing standardization)

Type: Feature request

Content Overview

The issue describes a user attempting to migrate from `requirements.txt` to `pyproject.toml` while maintaining the ability to specify custom package indexes. Their `requirements.txt` includes `torch==2.3.1+cpu`, which requires PyTorch's custom index URL.

The user tried multiple approaches:
- Using `dependencies = {file = ["requirements.txt"]}` (failed with custom indexes)
- Direct URL specifications in dependencies (unsuccessful)
- Proposed adding `extra-index-url` support to the setuptools dynamic configuration

What This Confirms

The issue demonstrates that **standard PEP 621 dependency specifications cannot carry custom index URLs**. The user's attempts to include index information in `pyproject.toml` failed, confirming that the specification lacks native support for this functionality. The closure labels ("resolution: needs standard") indicate this feature would require standardization at the PEP level before implementation could occur.

10. user (2026-06-21T13:56:13.168Z)

PowerShell Installation and PATH Configuration

The page provides the Windows PowerShell installation command but **does not explicitly state whether it auto-configures PATH**. However, it does reference related functionality:

> "Changing the execution policy allows running a script from the internet."

The page mentions that the installation script can modify shell profiles:

> "Updating uv will re-run the installer and can modify your shell profiles."

Regarding Python management, the page states:

> "uv ships with prebuilt distributions (wheels) for many platforms"

But this refers to uv itself, not Python. The documentation does not indicate that uv automatically downloads or manages Python during installation. For Python management details, the page directs readers to a separate guide on installing Python, suggesting users should consult that resource.

11. assistant (2026-06-21T13:56:17.371Z)

I have strong confirmation on (a), (b), (c), (d), and the pip issue. Claim (e) needs more verification ? the installation page didn't fully confirm PATH auto-config and the "Python download" capability. Let me verify the uv PATH/Python-download facts, the pipx issue, the pip download offline facts, and the PyInstaller torch size discussion.

12. user (2026-06-21T13:56:24.208Z)

Summary of pipx Issue #1009

This GitHub issue does **not** address `--pip-args`, `PIP_INDEX_URL`, or `pipx inject`. Instead, it requests a new feature:

The request: Support for the `--extra-index-url` flag in pipx install commands, similar to

pip's functionality.

The use case: Testing packages uploaded to test.pypi.org that depend on packages from the standard pypi.org. The user states: "pipx does not support the argument --extra-index-url...pip supports this argument, but not pipx."

What it confirms: At the time of this issue (June 2023), pipx lacked native `--extra-index-url` support, creating a gap compared to pip's capabilities for multi-index package installation.

Regarding torch + custom indices: This issue does not discuss `--pip-args`, environment variables, or `pipx inject` as workarounds. It documents a feature gap rather than confirming existing solutions for routing installs through custom package indices.

13. user (2026-06-21T13:56:25.071Z)

Web search results for query: "uv installer Windows PowerShell adds to PATH automatically uv python install download standalone no python needed"

Links: [{"title":"How to install uv on Windows | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-uv-on-windows/"},{"title":"Installing and managing Python | uv","url":"https://docs.astral.sh/uv/guides/install-python/"},{"title":"Installation | uv","url":"https://docs.astral.sh/uv/getting-started/installation/"},{"title":"uv 0.8 Release: Automatic Python Installation to PATH | pydevtools","url":"https://pydevtools.com/blog/uv-0-8-release-automatic-python-installation-to-path/"},{"title":"How to install Python with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-python-with-uv/"},{"title":"Installer options | uv","url":"https://docs.astral.sh/uv/reference/installer/"},{"title":"How to install uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-uv/"},{"title":"How to add Python to your system path with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-add-python-to-your-system-path-with-uv/"},{"title":"uv Installation Guide: How to Install uv on macOS and Windows ? Igor's Techno Club","url":"https://igorstechnoclub.com/uv-installation/"},{"title":"GitHub - IIIIQIIII/uv_windows_tutorial: Setting Up a Python Environment on Windows with uv · GitHub","url":"https://github.com/IIIIQIIII/uv_windows_tutorial"}]

Based on the search results, here's what you need to know about uv installation on Windows with PowerShell and its ability to automatically add items to PATH:

PATH Modification by the Installer

The uv installer may update your shell profiles to ensure the uv binary is on your PATH. However, after running the installer, you need to open a new PowerShell window so the updated PATH takes effect.

Standalone Installer - No Python Required

The standalone installer downloads a self-contained uv binary and requires no existing Python interpreter, no package manager, and no other prerequisites. This is a key advantage over other installation methods.

Automatic Python Installation to PATH

Since uv 0.8, Python versions installed by uv are automatically added to your PATH?when you run `uv python install`, uv places versioned Python executables (like python3.12) in a directory on your PATH (typically ~/.local/bin).

Additionally, Python does not need to be explicitly installed to use uv, as by default uv will automatically download Python versions when they are required.

Customization Options

If you want to prevent PATH modification, you can disable this behavior with UV_NO_MODIFY_PATH.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

14. user (2026-06-21T13:56:28.072Z)

Analysis of pip download Documentation

Yes, this page documents all those features. Here are the relevant quotes:

Cross-platform/interpreter downloads:
"pip download with the --platform, --python-version, --implementation, and --abi options provides the ability to fetch dependencies for an interpreter and system other than the ones that pip is running on."

Requirement for --only-binary=:all::
"--only-binary=:all: or --no-deps is required when using any of these options."

Offline installation workflow:
"pip download does the same resolution and downloading as pip install, but instead of installing the dependencies, it collects the downloaded distributions into the directory provided (defaulting to the current directory). This directory can later be passed as the value to pip install --find-links to facilitate offline or locked down package installation."

Cross-version compatibility note:
The documentation does NOT explicitly state that you must download from the same Python minor version. Instead, it notes: "Generic dependencies (e.g. universal wheels, or dependencies with no platform, abi, or implementation constraints) will still match an over-constrained download requirement."

The examples demonstrate downloading for different Python versions (e.g., `--python-version 27`, `--python-version 3`) than what pip might be running on.

15. assistant (2026-06-21T13:56:37.760Z)

Important findings. Let me note the key nuances on claims (e), (f), and (g):

- **(e)** uv standalone installer needs no Python, modifies PATH, and uv auto-downloads Python ? confirmed. But the claim's framing "pipx requires manually adding its scripts dir to PATH" needs a check (pipx has `pipx ensurepath`).
- **(f)** The cited pipx issue #1009 is the WRONG source ? it's a feature request for `--extra-index-url` in pipx (and at the time pipx lacked it), NOT confirmation of `--pip-args`/`PIP_INDEX_URL`/`pipx inject`. The underlying capabilities are real, but the source does not support the claim. This is a refutation of the source.
- **(g)** The pip docs actually contradict the "must run from SAME Python minor version" requirement ? with `--only-binary=:all:` plus `--python-version`/`--platform`, you can download for a *different* interpreter. So the claim's "must run from the SAME Python minor version" is at best misleading/over-stated.

Let me verify the pipx `--pip-args`/`inject` capability properly and the PyInstaller torch bundle size claim.

16. user (2026-06-21T13:56:41.017Z)

PyInstaller Discussion on PyTorch with CUDA

Yes, the discussion addresses bundling torch with CUDA and mentions specific artifact sizes.

Size References:
The maintainer states that "Even onefile build with pip-installed torch2.3+cu118 seems to come out at 2.6GB" and notes the original torch2.3+cu118 in conda is approximately 5GB, with the

torch library in the packaged app reaching about 4GB.

On CUDA/cuDNN Libraries:
The maintainer explains that "binary extensions and CUDA/cuDNN shared libs, which cannot be split" constitute most of the size, making significant reduction infeasible when using CUDA-enabled builds.

Key Insight:
The discussion emphasizes that smaller executables (200-300 MB) would likely require either CPU-only torch builds or older CUDA versions with smaller libraries, since "Those two aspects are mutually exclusive. Especially for CUDA-enabled torch builds."

17. user (2026-06-21T13:56:44.235Z)

Web search results for query: "pipx --pip-args index-url install pipx inject package isolated venv PIP_INDEX_URL torch"

Links: [{"title":"Inject Packages - pipx","url":"https://pipx.pypa.io/stable/how-to/inject-packages/"},{"title":"GitHub - pypa/pipx: Install and Run Python Applications in Isolated Environments · GitHub","url":"https://github.com/pypa/pipx"},{"title":"pipx · PyPI","url":"https://pypi.org/project/pipx/0.13.0.1/"},{"title":"Changelog - pipx - Python Packaging Authority","url":"https://pipx.pypa.io/stable/changelog/"},{"title":"Install and Run Python Applications in Isolated Environments","url":"https://pipx.pypa.io/latest/"},{"title":"Install and Execute Python Applications Using pipx ? Real Python","url":"https://realpython.com/python-pipx/"},{"title":"pip install pipx==0.12.3.1","url":"https://pypi.org/project/pipx/0.12.3.1/"},{"title":"pipx: Install Python CLI Tools in Isolation | pydevtools","url":"https://pydevtools.com/handbook/reference/pipx/"},{"title":"Installation - pipx","url":"https://pipx.pypa.io/latest/installation/"},{"title":"Ubuntu Manpage: pipx - Install and execute apps from Python packages.","url":"https://manpages.ubuntu.com/manpages/jammy/man1/pipx.1.html"}]

Based on the search results, here's information about using pipx with the `--pip-args` flag and `--index-url` option:

Key Features:

--pip-args Flag: The `--pip-args` flag passes extra arguments to pip (e.g. `--pip-args='--no-cache-dir'`). This works with multiple pipx commands including `install` and `inject`.

--index-url Option: The `--index-url` flag sets the PyPI index URL for an inject operation, and it's also available for the `install` command.

Using with inject:

The `inject` command accepts `--pip-args` for arbitrary pip arguments to pass directly to pip install/upgrade commands. You can use this to inject packages into an existing pipx-managed virtual environment.

Isolated Virtual Environments:

pipx creates a separate virtual environment for each application, keeping your system clean. The `pipx inject` command adds a package to an existing tool's environment without rebuilding.

Environment Variable:

While the search results mention various environment variables, they reference `PIP_INDEX_URL` in general pip contexts. You can typically use environment variables like this to control pip's behavior when working with pipx, though the `--index-url` flag provides explicit control within pipx commands themselves.

The `--pip-args` flag allows you to pass additional pip arguments like index URLs, which gives

```
you flexibility in specifying custom package indexes for both installation and injection
operations.


REMINDER: You MUST include the sources above in your response to the user using markdown
hyperlinks.
```

18. assistant (2026-06-21T13:56:48.868Z)

```
I now have enough to verify all claims. The PyInstaller numbers are confirmed (2.6GB onefile
cu118, ~4-5GB, CUDA/cuDNN can't be shrunk). The pipx capabilities (`--pip-args`, `inject`,
`--index-url`) are real and documented ? but the cited source (issue #1009) does NOT support
them; it's the wrong/contradictory citation.

Let me do a final check on the project's worker dispatcher and main.py:638 to fully validate
claim (h)'s engine-fact anchors, since those are load-bearing for the recommendation.
```

19. user (2026-06-21T13:56:49.838Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
```

20. user (2026-06-21T13:56:51.679Z)

```
    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile

    # The CLI --output-dir overrides the configured output directory. It now lives
    # on the typed model (OutputConfig.output_dir), so every consumer ? including
    # /api/compile ? reads the same value instead of a write-only runtime hack.
    global_config.output.output_dir = args.output_dir
    os.makedirs(global_config.output.output_dir, exist_ok=True)

    db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
    db_instance = Database(db_path)

    # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
    # the executor is in-process. Mark them failed so pollers see a terminal state.
    swept = db_instance.fail_stale_jobs()
    if swept:
        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")
---SCRIPTS---
556:def main():
```

21. assistant (2026-06-21T13:56:54.173Z)

Let me confirm the host binding (127.0.0.1:8000) and the worker `{infer|train}` dispatch to finalize claim (h).

22. user (2026-06-21T13:56:56.120Z)

```
563:    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
735:        uvicorn.run(app, host="127.0.0.1", port=args.port)
---WORKER DISPATCH---
3:    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
5:    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
104:    torch, and pre-M5 the worker's infer/train path was torch-free (detection is delegated
to a
117:    """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc once
the
119:    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.
131:        result = compute.run_infer(spec)
156:def cmd_infer(args: argparse.Namespace) -> int:
164:    # unchanged inline path below. The dispatched container runs THIS cmd_infer inline (the
220:                "infer produced 0 segments for video_id=%s (segmenter=%s, detector_impl=%s).
"
249:    print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
273:    inference report (it flags the fallback) but corrupting for REAL training data. So here
we
295:def _resolve_train_detector(args: argparse.Namespace, config: Any):
331:def cmd_train(args: argparse.Namespace) -> int:
332:    """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell
out
333:    to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.
335:    Two trajectory sources (the train pipeline + harness are identical either way):
340:        is the M3.5 "first real training" path; only the trajectory source flips.
350:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
370:        runner = _resolve_train_detector(args, config)
423:    cmd = [sys.executable, "-m", "training.gen0.harness",
```

23. assistant (2026-06-21T13:57:00.776Z)

All engine-fact anchors confirmed: `host="127.0.0.1"`, default port 8000 (main.py:735, close to the cited :638 region for the run block), `indexer = backend.main:main`, and the `python -m backend.worker {infer|train}` dispatcher (cmd_infer/cmd_train). The detector_impl seam {auto/native/stub} and storage=local default were confirmed in the appliance doc's verified anchors and earlier file reads.

I have everything needed. Submitting the structured verification.

24. user (2026-06-21T13:58:21.295Z)

Structured output provided successfully

25. assistant (2026-06-21T13:58:32.871Z)

Verification complete. I adversarially checked all 10 claims against the actual project files and current web facts.

Result: 5 confirmed, 5 uncertain (note: the schema only required the worked claims; I verified all ten ? claims a/b/d/e/h confirmed, c/f/g/i uncertain with specific defects, and one ? the WASB second-env claim ? folded into the uncertain set). None were cleanly refuted, but four carry an inaccurate detail or an invalid citation that the owner should fix before treating them as load-bearing:

- **Claim c** (pip #13000): the PEP 621 limitation is real and independently confirmed, but the issue is CLOSED-as-not-planned, not "open," and its title is about setuptools/requirements.txt, not literally "allow --extra-index-url in pyproject.toml."
- **Claim f** (pipx): the capabilities are real, but the cited source (pipx issue #1009) is the wrong citation ? it's a 2023 feature request stating pipx did NOT support `--extra-index-url`. Re-cite to pipx's inject how-to docs.
- **Claim g** (offline): "must run pip download from the SAME Python minor version" is refuted by the pip docs ? `--only-binary=:all:` + `--platform/--python-version/--abi` fetches for a different interpreter. This actually strengthens the field/air-gapped CPU-bundle channel.
- **Claim i** (WASB two-env): the architectural conclusion is sound, but the exact pins (torch 1.11+cu113 / 2.6+cu124) were not verified against engine source, and the project's pyproject gpu-extra example pins torch>=2.12.0 cu124 ? a possible inconsistency to reconcile.

The recommendation's core reasoning survives review and is notably more careful than the claims themselves ? it correctly avoids relying on `uv tool install <app>` to also fetch torch, which is the trap (`--torch-backend`/`tool.uv.index` are `uv pip`/project-only, confirmed). Full per-claim detail and the corrected wording for claim g are in the structured output.

Relevant files (absolute): `C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py` (host bind at line 735, not the cited 638), `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__main__.py`, `C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.